

TCP Improvements for High RTT Connections

Alexander Sukennyk

Department of Computer Science
Cleveland State University
Cleveland, Ohio, United States
a.sukennyk@vikes.csuohio.edu

Abstract—When transmission round trip times are large, the TCP protocol is slow in claiming bandwidth by adjusting its window size. This is due to fundamentally slow scaling in the standard TCP protocol. Once transmission has begun and “slow start” has ended, window size is incremented additively, but decremented multiplicatively when ACKs are not received. In non-noisy conditions, particularly in low congestion high RTT connections such as satellite transmissions, this approach trades efficiency for resilience when it is not necessary. This paper proposes and simulates alternative algorithms which could improve performance in these circumstances.

Keywords—TCP, Experiments, Simulation, Metrics, Networks

I. INTRODUCTION (HEADING 1)

Recent research has had a focus on improving TCP on transmission media with long propagation delays, causing long round trip times (RTTs). With the revival of interest in lunar research due to NASA’s Artemis program, it has become clear that the regular TCP protocol is insufficient for the long RTTs (>2.5s) of space communication. Other such examples exist in terrestrial circumstances as well, with communications between continents sometimes still operating with RTTs in the order of 1-2 seconds. These long RTT times reduce the rate at which window size scaling as the TCP congestion control algorithm only updates window size once feedback is returned from the receiver in the form of ACKs [2]. The rate at which bandwidth is claimed is therefore negatively correlated to RTTs, making regular TCP connections struggle to operate efficiently in these conditions.

A. TCP Congestion Control

Modern TCP connections use TCP Reno, an upgrade to TCP Tahoe. It rotates between 3 phases – slow start, congestion avoidance, and congestion detection. Each of these serve a specific purpose towards utilizing as much bandwidth as possible while working fairly with other connections using the same transmission medium [3].

At the start of a connection, TCP Reno engages its slow start. Each round trip, the window size is increased to 2^k where k is the number of round trips

completed since the beginning of slow start. If a threshold is reached, the window size is sent to the threshold and congestion avoidance (CA) begins. If a packet is not acknowledged, a threshold is set at the previous successful window size, and CA begins from there.

During congestion avoidance, the window size increases by 1 per RTT. If any fault occurs in transmission, the congestion detection phase kicks in. Congestion detection determines the thread of congestion depending on the type of fault. If 3 duplicate acknowledgements, or DUPACKs, are received, the threat of congestion is low, and the window is only decreased to half of its current size, then continues with the CA phase. If a timeout occurs, the threat of congestion is high, and the window is instead set to its initial size, restarting the process from the slow start phase.

This is the additive increase, multiplicative decrease method of TCP referred to as AIMD. Due to the restriction of one update per round trip, this method struggles to improve window size and enter its cyclic process quickly on high RTT communication lines. The aim of this paper is to show how this process can be improved.

B. Related Work

Authors Song, Zhang, and Sridharan created a TCP approach referred to as Compound TCP. The focus of their paper was creating methods which do not impede on the operations of standard TCP approaches. A metric introduced by them, bandwidth stolen, is used as an evaluation within this paper [1]. The 4th method tested in this paper is a simple approximation of their approach, labeled Approximated Compound TCP – ACTCP.

Authors Bhandarkar, Jain, and Reddy introduce a layered approach to congestion control named LTCP in their paper [4]. Their method scales TCP to high speed, high RTT networks by increasing initial bandwidth claimant quickly, with both macro and microscopic responses to conditions interpreted from ACKs read. They introduce a concept of a ‘compensation factor’ to correct for bandwidth which may have been stolen by their method.

Authors Dayma, Vanzara have proposed an improvement to TCP Hybla, TCP Hybla-i optimizing connections in high RTT satellite connections by reducing congestion window size to match bandwidth used effectively previously [5]. Method 1 within this paper is effectively a rough approximation of the Hybla-i process.

Work has been done to improve TCP for IoT, such as papers by Lim [6]; Talau, Herek, Fonseca, and Wille [7]. Their efforts have generated solutions which would not solve issues addressed in this paper, so they are out of the consideration scope.

A paper by Kachooei et. al. [8] proposes improvements to slow start performance. Their method ramps transmission rates quickly but does not maintain TCP friendliness due to its lack of ability to sync with regular TCP connections. Noting this issue, this paper makes no effort to improve on the slow start phase of TCP, to prioritize TCP friendliness without compensation efforts.

II. EVALUATION METRICS

Several metrics can be used to evaluate improvement methods. The most important of these are bandwidth utilization, retransmission frequency, and TCP friendliness. An ideal improvement to the current TCP protocol would maintain the same retransmission rate and TCP friendliness as TCP Reno, while increasing bandwidth utilization. For each experiment in this paper these evaluation metrics will be used for analysis. Assumptions relevant to how these metrics are calculated are described in further detail in *Section IV.A*.

For a 1:1 sender-receiver transmission, bandwidth utilization is computed as follows: Given a list M containing the maximum bandwidth a receiver can handle at each time interval (step) t , and a list W containing the bandwidth effectively utilized at each of n steps, bandwidth utilization can be expressed as:

$$B_U = 1 - \frac{1}{n} \sum_{t=0}^n \frac{(M_t - W_t)}{M_t}$$

For a transmission with a sender-receiver ratio $S:I$; $S > I$, the bandwidth utilization for any sender S_i is interpreted as a percentage of their “fair share”. Assuming that for S senders the fair share of total bandwidth for each is $\frac{1}{S}$, bandwidth utilization for a single sender is:

$$B_{U,i} = 1 - \frac{1}{n} \sum_{t=0}^n \frac{(\frac{1}{S}M_t - W_{t,i})}{\frac{1}{S}M_t} = 1 - \frac{1}{n} \sum_{t=0}^n \frac{(M_t - SW_{t,i})}{M_t}$$

This utilization can exceed a ratio 1, indicating that a particular sender has utilized more than their fair share. A sender with bandwidth utilization in excess of 1 has “stolen” bandwidth from other senders. A formula to calculate the percentage of

bandwidth stolen is provided by authors Song, Zhang, and Sridharan:

$$B_{stolen} = \frac{P-Q}{P} [I]$$

Here P is throughput when m regular TCP flows compete with l regular TCP flows, and Q is throughput for m regular TCP flows competing with l high-speed flows. Hence, each experiment will need to be compared with a control run to see the effects of each method on regular connections sharing the same medium. If stolen bandwidth is in excess of 10%, the suggested high-speed method is evidently not TCP friendly.

Retransmission frequency in this paper will be expressed as a number of faults experienced per 100 steps. Under the assumption that the retransmission of a window of a whole is required in the case of a fault, this frequency is related to packet loss by a scalar factor, justifying this approach. An increase in retransmission frequency indicates a higher packet loss regardless of whether this assumption is true or not, and therefore it is in both cases an effective metric to observe changes in stability for a particular algorithm.

III. PROPOSED ALGORITHMS

Each of the approaches tested makes some adjustment to the AIMD model used in regular TCP. Due to the requirement of TCP friendliness for these general purpose improvement attempts, actions which increase the speed of slow start would result in fast over-claiming that would not be permitted by most receiving protocols. The concept of adjusting slow start instead is described in section 7.

When referenced past their section, each model will be referred to as “Method #” or “M#”.

A. Method 1 – Decrease Less

The Decrease Less approach is simple in nature, but has the potential to reduce wasted bandwidth. Rather than following the AIMD model with a decrease by a factor $F = 1/2$, instead we use some factor $F > 1/2$. When 3 DUPACKs are received by the sender, the window size is decreased to this factor instead, and the additive increase is then stalled for $ws_{prev} * (F - 1/2)$ round trips. This stall allows the window size to hold steady until it matches up with regular TCP connections. This effect is shown in Fig. 1.

As the value of F increases, this method becomes less TCP friendly, but increases its bandwidth utilization. If the 3 DUPACK trigger for decrease happens multiple times in a row, this alternative protocol would decrease more slowly than regular TCP connections, hogging more bandwidth for itself after the congestion clears up. This is an issue, as a

receiver will timeout a sender that deliberately “skips the line” in this manner.

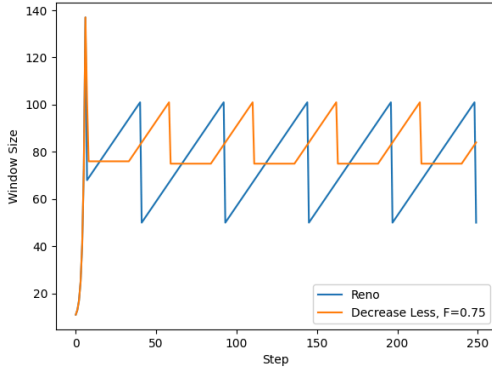


Fig. 1: Decrease Less method compared to Reno in 2 separate 1:1 connections with no congestion.

B. Method 2 – Greedy Grabbing

The greedy grabbing approach increases window size multiplicatively after the DUPACK failure condition. When the window size is cut in half after this fault, this algorithm increases the window by $\max[\frac{1}{2}(\text{threshold} - \text{window size}), 1]$. This multiplicative increase replacing the additive increase for a brief moment results in very fast, but unfriendly bandwidth claiming. This algorithm has been designated “Greedy Grabbing” because it aggressively steals bandwidth from normal TCP connections. This would not be permitted by an open receiver. Fig. 2. shows the graph for this method.

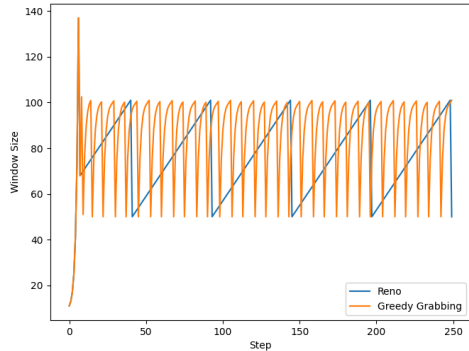


Fig 2: Capped Greed method compared to Reno in 2 separate 1:1 connections with no congestion.

It is clear that in addition to its lack of TCP friendliness, many more timeouts are experienced by this method when it operates alone. In this way, despite its increase in apparent bandwidth utilization, it may be worse than TCP Reno is to begin with. However, unlike method 1, greedy grabbing does not reduce the amount the window size is decreased by for DUPACK faults. Therefore, this method will not reserve extra space for itself after multiple DUPACK faults. Method 3 expands on both of these methods to merge their benefits.

C. Method 3 – Capped Greed

The capped greed method uses the same adding-halves approach as method 2, but approaching a percentage of the threshold rather than the threshold. Once the window size reaches this value, window size increases are halted until the standard TCP slope catches up, as in method 1. This effect compared to TCP Reno is shown in Fig. 3.

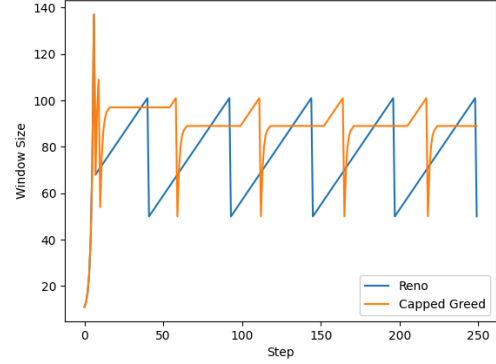


Fig. 3: Capped Greed method compared to Reno in 2 separate 1:1 connections with no congestion.

D. Method 4 – ACTCP

The paper by Song, Zhang, and Sridharan [1] introduces Compound TCP, an approach which uses exponential increase until some upper threshold is reached. By modifying Method 3, an approximation of this CTCP can be made, which will be referred to as Approximated Compound TCP for testing. This model performs fast recovery after congestion by growing exponentially as in slow start up to some cap. This cuts into the unused bandwidth zones less than capped greed, but is likely more TCP friendly on average, especially in noisy conditions.

IV. EXPERIMENTS

A. Experimental Assumptions

For each experiment, some maximum receiver bandwidth as a function of time is established. A series of steps are executed with each step representing a segment of time equal to the RTT. The size of any transmission is represented as a window size, rather than number of bits or packets. This reflects an assumption that any fault results in a computational expense equal to loss of the entire window. In a way, this overestimates loss rather than underestimating it.

Each of these assumptions are chosen to frame the problem in a manageable way without a need to compute specifics down to the bit level. The nature of the defined objects is such that this could be easily converted by adjusting the “resolution” of the simulations, and updating increments to be in terms of $\text{packet size} * \text{\#packets} = \text{\#bits}$ instead.

B. Isolated Exposure to Other TCP

A 4-sender single receiver simulation is run for each improvement method, where one sender uses one of the 4 described methods, competing for bandwidth with 3 senders using stock TCP Reno. 250 Steps are taken with only DUPACK faults. No noise is introduced. Maximum packets per RTT fixed at 99.

Parameters used:

- Decrease Less : $F = 0.95$
- Capped Greed : $\%thresh = 0.95$
- ACTCP : $\%thresh = 0.95$

Results are shown in Table I.

TABLE I: Results of Isolated Exposure Experiment for each Model (M1-M4), Compared to Regular TCP (Reno).

Model	Reno	M1	M2	M3	M4
$F_{retransmit}$	7.2	7.6	12.8	7.6	7.6
$B_{util} \%$	75.44	97.70	186.3	96.54	92.59
$B_{stolen} \%$	n/a	7.82	46.95	7.67	7.41

Table I's results reflect the expected behaviors of the models as described in *Section III*. Greedy grabbing has the worst TCP friendliness by far with 46.95% of bandwidth stolen from each Reno sender. The results of Models 1, 3 change as their parameters are adjusted. The following *Section C* addresses this.

C. Varying Parameters — Models 1,3

The results of Models 1, 3, and 4 vary with their parameters (F , $\%thresh$). To find effective parameters, we repeat the experiment from IV.B with each of these models, varying their parameters from values 0.5 to 1.5 with a step size of 0.01. Plotting the resulting retransmission loss rate, bandwidth utilization, and bandwidth stolen reveals the nature of these algorithms. Each of these is expected to have a massive inflection point when the parameter(s) are set to values greater than 1, as this is where overclaiming would begin. Plots are presented in Figs. 4, 5, and 6.

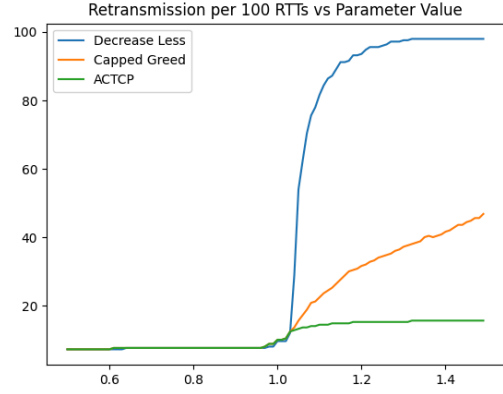


Fig. 4: Rate of retransmission comparison for Models 1, 3, and 4 with respect to parameter adjustment.

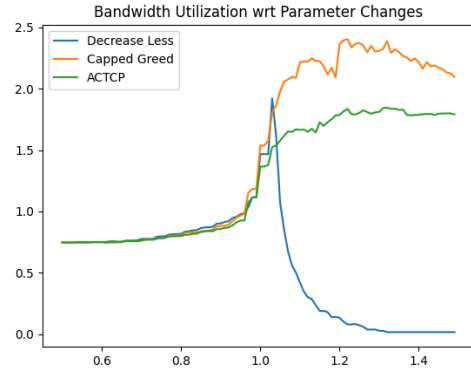


Fig. 5: Comparison of bandwidth utilization % for Models 1, 3, and 4 with respect to parameter adjustment.

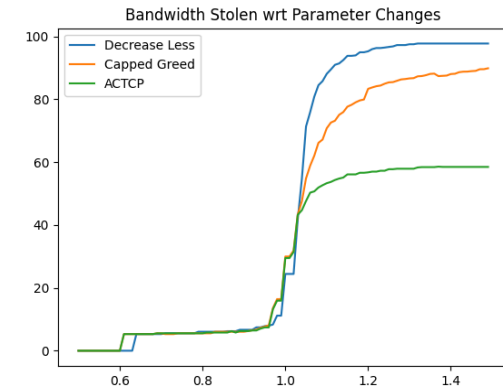


Fig. 6: Comparison of Models 1, 3, and 4 stolen bandwidth % with respect to parameter adjustment.

D. Exposure with Noise and Congestion

The IV.B experiment is repeated once more, with a modified receiver. This receiver adds noisy variance to the maximum packets it can receive, with a spread of ± 25 packets. The receiver has a 10% chance to time out completely and decrease the average max

packet by 10%. The step count is set to 1000 and initial $max_packets$ variable is set to 400. All other parameters remain the same.

This experiment further explores the effectiveness of the proposed methods when operating in conditions closer where congestion constricts communication progressively. The results are displayed in Table II.

Table II: Results of Isolated Exposure Experiment Repeated with Simulated Congested Transmission Medium.

Model	Reno	M1	M2	M3	M4
$F_{retransmit}$	3.6	20.7	37.9	4.1	3.9
$B_{util} \%$	70.72	83.87	92.74	99.30	99.74
$B_{stolen} \%$	n/a	30.56	63.52	11.43	10.53

In noisy conditions, the decrease-less method (M1) fails to maintain its friendliness and claims more than its fair share quickly. Despite this, it fails to keep up even with regular TCP in terms of bandwidth utilization.

E. Late Join

The final experiment simulates each proposed model joining transmission line with existing connections on it, analyzing the amount of bandwidth it can claim while competing and syncing with existing regular TCP connections. 1000 steps are simulated on a clean transmission medium. The test model joins after 100 steps have been completed, and its performance for the remaining 900 steps is considered. Table III shows the results.

Table III: Results of Late Join Experiment Repeated with Ideal Clean Transmission Medium.

Model	Reno	M1	M2	M3	M4
$F_{retransmit}$	7.25	7.25	13.0	7.25	7.25
$B_{util} \%$	70.15	85.79	187.44	84.27	81.68
$B_{stolen} \%$	n/a	0	47.901	0	0

V. RESULTS ANALYSIS

The experiments described in Section IV provide average-case results to use for analysis. This section will reference these results, as well as other observations made throughout the course of testing using these experiments. Additional window size vs. time graphs related to each test are provided as a visual aid in Appendix A.

Method 2 was included in this paper as a baseline least-friendly example. In each experiment, it claimed the most bandwidth aggressively with no regard for the other connections. Not much is learned regarding it in the experiments besides confirming its certain unfitness to the goal of this paper. It will not be specifically addressed moving forwards as there is no appropriate use of it in practice.

Experiment B demonstrated the results of each model in idealized conditions, where all senders establish a connection simultaneously on a noiseless medium. Method 1 is shown to function best under these conditions, with an acceptable stolen bandwidth percentage $<10\%$. Method 2 claims the most bandwidth, aggressively, but is only included in these comparisons as a worst-case scenario, as any receiver would deliberately timeout this method's aggressive approach. Methods 3 and 4 perform similarly, with Method 3 squeezing out 4% more bandwidth without dramatically increasing its stolen bandwidth percentage.

Experiment C showed how the metrics changed as the parameters for Methods 1, 3, and 4 were changed. At a parameter value of $P=0.5$, these methods reduce to standard TCP Reno, generating no unique benefits. Once a value $P=1.0$ is reached, the threshold adjustment that occurs for these methods exceeds the established threshold during the multiplicative decrease. Any P value equal to or greater than 1.0 makes these methods unfriendly to other TCP connections and should be avoided. Within the domain $\{0.9 < P < 1.0\}$, the methods have experienced an exponential growth in bandwidth utilization, while keeping stolen bandwidth and retransmission frequency within reasonable bounds. Hence, for any P within this domain, we can expect notable improvements to bandwidth claiming without many adverse effects.

Experiment D demonstrated the effects of a very noisy transmission medium and very congested receiver. In these conditions, Method 1 fails to uphold its TCP friendliness. This is due to the possibility of repeated DUPACK faults, during which the difference in threshold between senders using Method 1 and ones using TCP Reno increases with each sequential fault. The advantage of Methods 3 and 4 is their initial multiplicative decrease will match that of connections using TCP Reno. Rather than trying to keep a high threshold, like Method 1, these Method 3 and 4 perform better and are more TCP friendly in congested systems due to their increment being the only changed mechanism.

Experiment E showed results of each proposed method being introduced to an existing connection. With the methods being the new connection to the system, they do not have control over the synchronization, and are unable to greedily overclaim in any way, with the exception of Method 2. Of the

others, Method 3 managed to utilize the most of its ‘fair share’ of bandwidth. Both it and Method 4 made significant improvements by claiming an additional 9.27% and 6.68% bandwidth, respectively, without stealing from the other protocols at all. This observation justifies these methods as viable approaches to claiming extra bandwidth in existing connections. Data from an experiment with the noisy receiver was not included due to its messiness and a creep in project scope, but preliminary testing indicated that friendliness is upheld even in a messy and congested network.

VI. CONCLUSION

The results of the presented experiments reflect a high possibility that a simple adjustment to the congestion avoidance phase, such as the ones proposed in Methods 3 and 4, can be impactful in increasing bandwidth utilization on existing high RTT networks. By maximizing throughput in this way, the amount of ‘dead’ bandwidth in a connection is reduced, getting more value out of precious and expensive round trips. By allowing the modified TCP protocols to more quickly recover after a fault, rather than attempting to change the slow start or decrease phases, these two methods manage to achieve these improvements without stealing bandwidth from regular TCP connections.

Within the scope defined in these experiments, it appears that Method 3, a capped greedy approach, performs the best in both congested and uncongested conditions. It is likely that if Methods 3 and 4 were fully integrated in a physical sender, they would perform similarly, with the CTCP being more friendly at a small cost of slightly slower bandwidth claiming. I believe that both of these would be strong contenders for long distance communication with round trip times in excess of 2s, with more improvements being possible depending on the specialization of the networks.

VII. FUTURE WORK

A more accurate model must be generated to conclusively decide if these methods are successful in claiming additional bandwidth without stepping on the toes of existing regular TCP connections. By reducing the resolution, and generating simulations within a program such as ns-3, we could verify the correlation of these estimated results to reality. In doing so, it would also be significantly easier to make noisy network simulations close to real conditions to verify that these methods are resilient to noise and congestion as intended, and that these features of congestion control have not been lost with the proposed changes.

This paper focused mainly on modifying the congestion avoidance phase of the TCP congestion control algorithm. This is to allow the proposed

algorithms to mesh with and synchronize with other senders using the regular TCP protocol provided that the replacement to the CA phase is relatively TCP-friendly. In the introduction, the possibility of using alternative protocols for space communication is briefly mentioned. In the case of specialized transmission media such as this, there is a high likelihood that all transmissions are sent from a single controller at a given instance, in which case several doors are opened for overhauling the TCP congestion control algorithm entirely. In fact, it is possible that the slow start phase could be modified to be a much faster increase, such as a factorial one, to more greedily claim bandwidth. Future work around this topic may also consider broadening the horizons of transmission past TCP as a whole, opting for an entirely new, specialized protocol for these fringe use cases.

REFERENCES

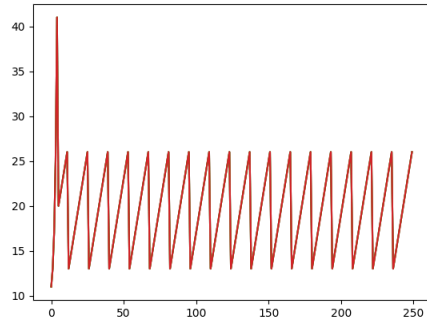
- [1] K. Tan, J. Song, Q. Zhang, and M. Sridharan, “A compound TCP approach for high-speed and long distance networks,” Proceedings IEEE INFOCOM 2006. 25TH IEEE International Conference on Computer Communications, 2006. doi:10.1109/infocom.2006.188
- [2] R. M. Long, “Evaluation of TCP congestion control algorithms,” Evaluation of TCP congestion control algorithms. (Technical Report) | OSTI.GOV, <https://www.osti.gov/biblio/918389>
- [3] [1] W. Stevens, *TCP slow start, congestion avoidance, fast retransmit, and fast recovery algorithms*, Jan. 1997. doi:10.17487/rfc2001
- [4] Bhandarkar, Sumitha , et al. *Improving TCP Performance in High Bandwidth High RTT Links Using Layered Congestion Control*. 2005, citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=705f348a2137feca43d70fd9e54e4a54fdc76e3a.
- [5] Reshma Dayma, and Rakesh Vanzara. “Improved TCP Hybla: A TCP Enhancement for Link with High RTT and Error Rate.” *IEEE*, vol. 2582, 1 Dec. 2012, pp. 1–6, [ieeexplore.ieee.org/abstract/document/6493205](https://doi.org/10.1109/nuicone.2012.6493205), <https://doi.org/10.1109/nuicone.2012.6493205>. Accessed 7 Dec. 2024.
- [6] Lim, Chansook. “Improving Congestion Control of TCP for Constrained IoT Networks.” *Sensors*, vol. 20, no. 17, 24 Aug. 2020, p. 4774, <https://doi.org/10.3390/s20174774>. Accessed 18 Nov. 2020.
- [7] Talau, Marcos, et al. “Improving TCP Performance over a Common IoT Scenario Using the Early Window Tailoring Method.” *Computer Networks*, vol. 234, 1 Oct. 2023, p. 109875, www.sciencedirect.com/science/article/abs/pii/S1389128623003201?via%3Dihub, <https://doi.org/10.1016/j.comnet.2023.109875>. Accessed 14 Apr. 2024.
- [8] Kachooei, Maryam Ataei, et al. “Improving TCP Slow Start Performance in Wireless Networks with SEARCH.” *IEEE*, 4 June 2024, [ieeexplore.ieee.org/abstract/document/10579174](https://doi.org/10.1109/wowmom60985.2024.00051), <https://doi.org/10.1109/wowmom60985.2024.00051>. Accessed 7 Dec. 2024.

APPENDIX A

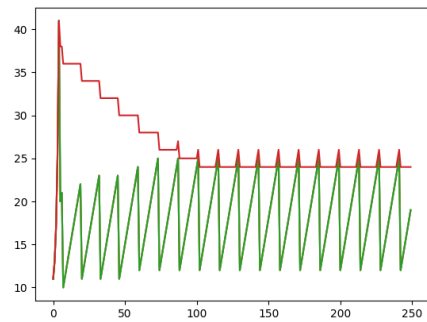
Below are the graphs showing window size as a function of time for each experiment, labeled. These contain a lot of important information, but could not be included within the text while keeping it brief.

EXP VI.B GRAPHS

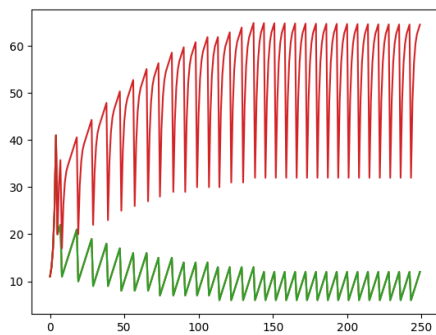
Reno vs. Self:



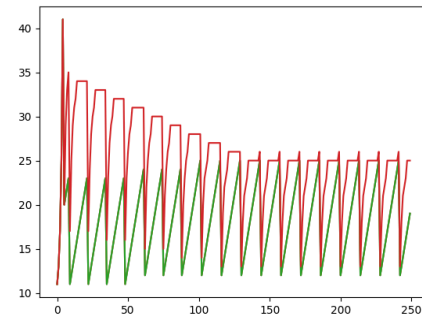
Reno Vs. Decrease Less:



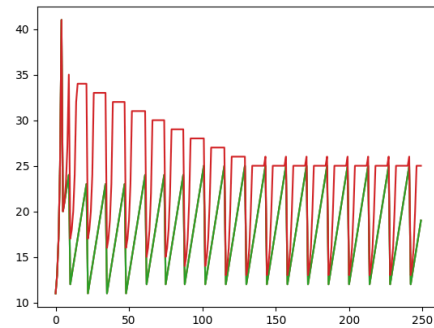
Reno vs. Greedy Grabbing:



Reno vs. Capped Greed:

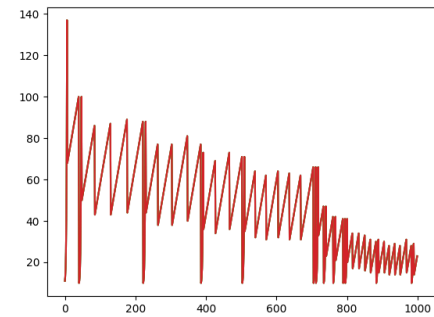


Reno vs. ACTCP:

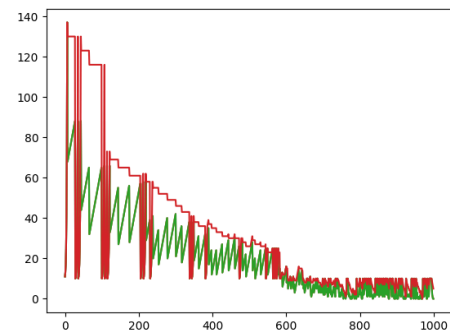


EXP VI.D GRAPHS

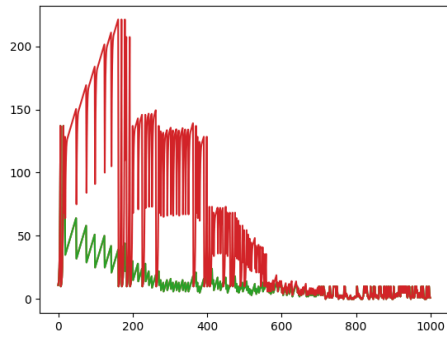
Reno vs. Self:



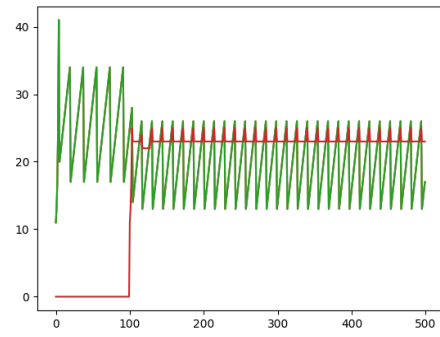
Decrease Less vs. Reno:



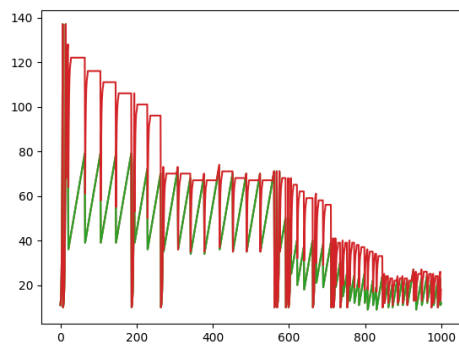
Greedy Grabbing vs. Reno:



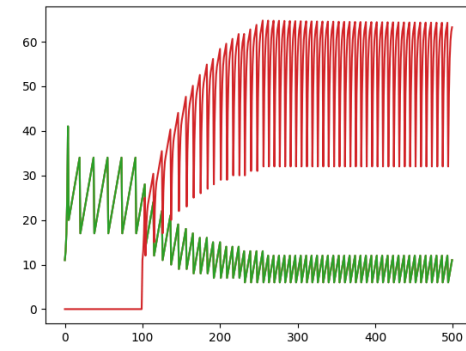
Decrease Less vs. Reno:



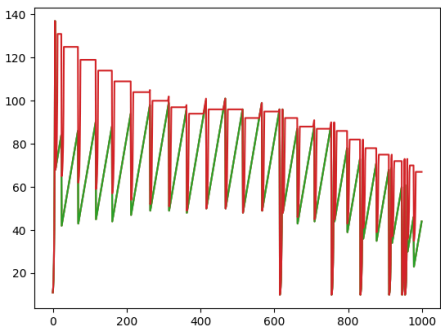
Capped Greed vs. Reno:



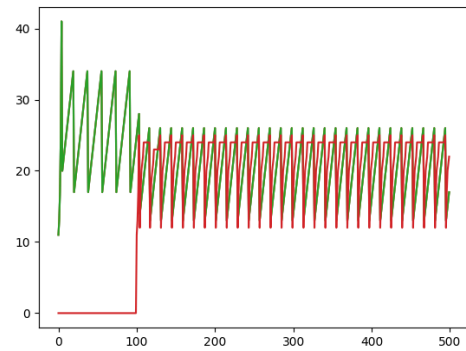
Greedy Grabbing vs. Reno:



ACTCP vs. Reno:

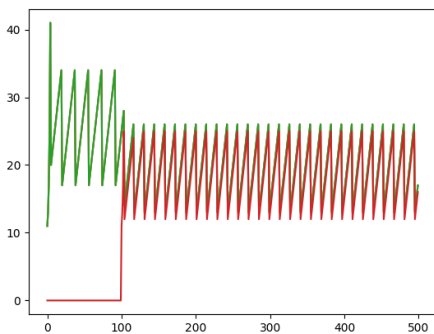


Capped Greed vs. Reno:



EXP V.I.E GRAPHS

Reno vs. Self:



ACTCP vs. Reno:

